UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO

PEDRO CABRAL GOMES

LISTA DE EXERCÍCIOS 2

Questões 1 a 5

MACEIÓ
2026

PEDRO CABRAL GOMES

LISTA DE EXERCÍCIOS 2

Questões 1 a 5

Trabalho apresentado à disciplina de Inteligência Artificial do Instituto de Computação da Universidade Federal de Alagoas, como requisito parcial para obtenção de nota.

Professor: Evandro Costa

MACEIÓ

2026

SUMÁRIO

1 QUESTÃO 1 – ONTOLOGIAS E GRAFOS DE CONHECIMENTO 4
1.1 Sistemas baseados em ontologias 4
1.2 Sistemas baseados em grafos de conhecimento 4
1.3 Exemplo de aplicação 5
1.4 Comparativo 6
1.5 Mapeamento de uma ontologia em um grafo de conhecimento 6
2 QUESTÃO 2 – ALGORITMO DOS K-VIZINHOS MAIS PRÓXIMOS (KNN) . . . 10
2.1 Base de dados 10
2.2 Preparação dos dados 11
2.3 Medida de distância 12
2.4 Passos do algoritmo 12
2.5 Escolha do valor de k 14
2.6 Avaliação 14
3 QUESTÃO 3 – O CHATBOT ELIZA 16
3.1 Descrição e principais funcionalidades 16
3.2 Trecho de diálogo 17
3.3 Partes tecnicamente interessantes 17
3.4 Partes tecnicamente falhas 18
3.5 Regras e algoritmo 19
3.6 Evolução: PARRY 20
3.7 Evolução: AIML e ALICE 21
4 QUESTÃO 4 – TIPOS DE AGENTES INTELIGENTES 23
4.1 Propriedades dos ambientes de tarefa 23
4.2 Agente reativo simples 23
4.3 Agente reativo baseado em modelo 24
4.4 Agente baseado em objetivo 25
4.5 Agente baseado em utilidade 26

4.6 Agente adaptativo 27
4.7 Síntese 28
5 QUESTÃO 5 – INFERÊNCIA EM GRANDES MODELOS DE LINGUAGEM . . 30
5.1 O vídeo e sua ideia central 30
5.2 Treinamento e inferência 30
5.3 Motor de inferência 31
5.4 Formato da requisição 31
5.5 Tokenização 31
5.6 Autenticação e roteamento 32
5.7 Agrupamento de requisições 32
5.8 Fase de prefill 33
5.9 Fase de decode 33
5.10 Métricas de latência 34
5.11 Síntese do processo 35
REFERÊNCIAS 36

1 QUESTÃO 1 – ONTOLOGIAS E GRAFOS DE CONHECIMENTO

1.1 Sistemas baseados em ontologias

Uma ontologia é uma especificação explícita de uma conceitualização (Gruber, 1993). Em termos simples, é uma descrição formal dos conceitos de um domínio, das relações entre eles e das regras que eles devem obedecer.

Uma ontologia tem três tipos de elementos. As classes representam os conceitos, como Professor e Disciplina. As propriedades representam as relações entre indivíduos, como ministra, e os atributos, como nome. Os axiomas são as afirmações lógicas sobre classes e propriedades, por exemplo: todo professor é uma pessoa, ou toda disciplina é ministrada por exatamente um professor. Os indivíduos, como a professora Helena, são os membros das classes.

É comum separar a ontologia em duas partes. A TBox guarda o esquema, formado por classes, propriedades e axiomas. A ABox guarda os fatos sobre os indivíduos.

A linguagem padrão para escrever ontologias é a OWL 2, recomendada pelo W3C (W3C OWL Working Group, 2012). Ela se baseia nas lógicas de descrição, uma família de lógicas em que o raciocínio é decidível. Por isso existem programas, chamados de *reasoners*, que fazem inferências automáticas sobre a ontologia. Os três serviços principais são: verificar se a ontologia tem contradições, descobrir relações de subclasse que não foram declaradas e classificar os indivíduos nas classes a que pertencem.

Uma característica importante é a hipótese do mundo aberto. Em OWL, o que não está declarado não é considerado falso, é considerado desconhecido. Se não está escrito que Helena orienta alguém, não se pode concluir que ela não orienta ninguém.

1.2 Sistemas baseados em grafos de conhecimento

Um grafo de conhecimento é um grafo de dados em que os nós representam entidades de interesse e as arestas representam relações entre essas entidades (Hogan *et al.*, 2021). A unidade básica é a tripla, formada por sujeito, predicado e objeto, como em (helena, ministra, ia).

Há dois modelos mais usados. No modelo RDF, tudo é tripla e as consultas são feitas na linguagem SPARQL. No modelo de grafo de propriedades, usado por bancos como o Neo4j, os nós e as arestas têm rótulos e podem guardar atributos, e as consultas são feitas em linguagens como a Cypher.

O grafo de conhecimento costuma ser construído a partir de muitas fontes: bancos de dados, planilhas, textos e páginas da web. O esquema é flexível. Ele pode ser uma ontologia completa, uma lista simples de tipos ou nem existir de forma explícita. Novos tipos de nós e de arestas podem ser acrescentados sem mudar o que já existe.

A inferência em grafos de conhecimento acontece de três formas. A primeira é a consulta por caminhos, que segue várias arestas para chegar a uma resposta. A segunda é a aplicação de regras, que geram triplas novas a partir das existentes. A terceira é o aprendizado de máquina: modelos de *embeddings* e redes neurais em grafos estimam ligações que faltam. Essa terceira forma produz conclusões prováveis, e não conclusões garantidas.

1.3 Exemplo de aplicação

O exemplo é o sistema acadêmico de uma universidade, com professores, alunos, disciplinas e projetos de pesquisa.

Em um sistema baseado em ontologia, o trabalho começa pelo esquema. Define-se que Professor e Aluno são subclasses de Pessoa, que ministra liga um professor a uma disciplina, que toda disciplina tem exatamente um professor e que só um professor com doutorado pode orientar alunos de doutorado. Depois os dados são inseridos e o *reasoner* é executado. Ele consegue, por exemplo, classificar automaticamente quem é orientador de doutorado e acusar uma inconsistência se uma disciplina for cadastrada com dois professores. Esse tipo de sistema é indicado quando as regras do domínio são importantes e os dados precisam ser validados, como na conferência de uma oferta de disciplinas.

Em um sistema baseado em grafo de conhecimento, o trabalho começa pelos dados. Os registros do sistema de matrícula, da plataforma de currículos e do repositório de publicações são carregados como nós e arestas. O grafo resultante pode ter milhões de triplas. As perguntas típicas são de navegação: quais professores publicaram com ex-alunos, quais disciplinas costumam ser cursadas juntas, que disciplina recomendar a um aluno. Esse tipo de sistema é indicado quando há muitos dados de fontes diferentes e o objetivo é consultar, explorar e recomendar.

Na prática, as duas abordagens se combinam. A ontologia serve de esquema para o grafo, e o grafo guarda os dados em grande volume.

1.4 Comparativo

O Quadro 1 resume as diferenças entre as duas abordagens.

Quadro 1 – Comparativo entre sistemas baseados em ontologias e em grafos de conhecimento

Aspecto	Ontologias	Grafos de conhecimento
Foco	Significado dos conceitos e regras do domínio	Dados sobre entidades e suas ligações
Ponto de partida	Esquema (classes, propriedades e axiomas)	Dados de várias fontes
Esquema	Obrigatório e rígido	Opcional e flexível
Linguagens	OWL 2, RDFS	RDF e SPARQL; grafo de propriedades e Cypher
Raciocínio	Dedutivo, feito por *reasoner*, com garantias lógicas	Consultas por caminhos, regras e aprendizado de máquina
Suposição sobre o que falta	Mundo aberto: o que não está dito é desconhecido	Em geral mundo fechado nas consultas: o que não está no grafo não é retornado
Tamanho típico	Centenas ou milhares de classes e poucos indivíduos	Milhões ou bilhões de triplas
Desempenho	Raciocínio caro em bases grandes	Consultas rápidas em bases grandes
Qualidade dos dados	Inconsistências são detectadas	Dados incompletos e com ruído são tolerados
No exemplo da universidade	Validar a oferta e classificar quem pode orientar	Integrar matrícula, currículos e publicações; recomendar disciplinas

Fonte: elaborado pelo autor (2026).

1.5 Mapeamento de uma ontologia em um grafo de conhecimento

Mapear uma ontologia em um grafo de conhecimento significa escrever cada elemento da ontologia como nós e arestas. Para o modelo RDF esse mapeamento é padronizado pelo W3C (Patel-Schneider; Motik, 2012). O Quadro 2 mostra as regras principais e a forma equivalente no modelo de grafo de propriedades.

Quadro 2 – Regras de mapeamento dos elementos da ontologia

Elemento da ontologia	No grafo RDF	No grafo de propriedades
Classe	Nó ligado a owl:Class por uma aresta rdf:type	Rótulo de nó
Indivíduo	Nó	Nó
Indivíduo pertence a uma classe	Aresta rdf:type do indivíduo para a classe	Rótulo colocado no nó
Subclasse	Aresta rdfs:subClassOf entre os nós das classes	Aresta entre nós de esquema ou rótulos adicionais no nó
Propriedade de objeto entre dois indivíduos	Aresta com o nome da propriedade	Aresta com o tipo da propriedade
Propriedade de dados	Aresta do indivíduo para um valor literal	Atributo guardado dentro do nó
Domínio e imagem	Arestas rdfs:domain e rdfs:range saindo do nó da propriedade	Em geral ficam fora do grafo, como restrições do banco
Axiomas complexos (restrições)	Várias triplas com nós auxiliares	Não são representados; seus efeitos entram como dados inferidos

Fonte: elaborado pelo autor (2026).

O exemplo usa uma parte pequena da ontologia da universidade. Na TBox, Professor e Aluno são subclasses de Pessoa, e a propriedade ministra tem domínio Professor e imagem Disciplina. Na ABox, helena é uma professora de nome Helena Souza, joao é um aluno, ia é uma disciplina, helena ministra ia e joao cursa ia.

O mapeamento é feito em quatro passos.

Passo 1. Mapear o esquema. Cada classe e cada propriedade vira um nó, e cada axioma simples vira uma aresta. O axioma "Professor é subclasse de Pessoa" vira a tripla (Professor, rdfs:subClassOf, Pessoa).

Passo 2. Mapear os indivíduos. Cada indivíduo vira um nó, ligado à sua classe por uma aresta rdf:type. Cada relação entre indivíduos vira uma aresta com o nome da propriedade. Cada atributo vira uma aresta para um valor.

Passo 3. Materializar as inferências. O *reasoner* é executado sobre a ontologia e as conclusões são gravadas no grafo como triplas comuns. No exemplo, como helena é Professor e Professor é subclasse de Pessoa, grava-se (helena, rdf:type, Pessoa). Assim o grafo responde à consulta "quais são as pessoas" sem precisar raciocinar na hora.

Passo 4. Carregar o resultado em um banco de grafos, onde ele pode ser consultado e ampliado com dados de outras fontes.

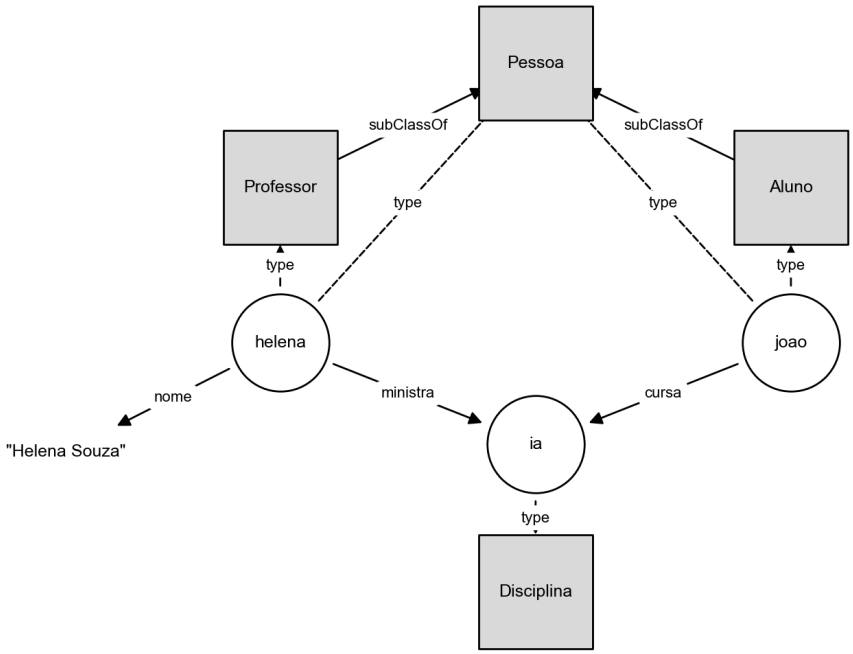
O Quadro 3 mostra as triplas geradas e a Figura 1 mostra o grafo. Para a figura ficar legível, as duas triplas de domínio e imagem da propriedade ministra não foram desenhadas.

Quadro 3 – Triplas resultantes do mapeamento

Origem	Sujeito	Predicado	Objeto
Esquema (TBox)	Professor	rdfs:subClassOf	Pessoa
Esquema (TBox)	Aluno	rdfs:subClassOf	Pessoa
Esquema (TBox)	ministra	rdfs:domain	Professor
Esquema (TBox)	ministra	rdfs:range	Disciplina
Fatos (ABox)	helena	rdf:type	Professor
Fatos (ABox)	joao	rdf:type	Aluno
Fatos (ABox)	ia	rdf:type	Disciplina
Fatos (ABox)	helena	ministra	ia
Fatos (ABox)	joao	cursa	ia
Fatos (ABox)	helena	nome	"Helena Souza"
Inferência	helena	rdf:type	Pessoa
Inferência	joao	rdf:type	Pessoa

Fonte: elaborado pelo autor (2026).

Figura 1 – Grafo de conhecimento obtido a partir da ontologia (arestas tracejadas são inferidas)



Fonte: elaborado pelo autor (2026).

No grafo, o esquema e os dados ficam juntos: as classes aparecem como nós, ao lado dos indivíduos. As arestas tracejadas não foram declaradas por ninguém. Elas vieram do raciocínio sobre a ontologia e passaram a fazer parte dos dados.

No modelo de grafo de propriedades o resultado seria mais compacto. O nó helena teria os rótulos Professor e Pessoa e o atributo nome dentro dele, e haveria

uma aresta do tipo MINISTRA até o nó ia.

O mapeamento tem um limite. Axiomas como "toda disciplina tem exatamente um professor" podem ser gravados como triplas, mas o banco de grafos não os entende como regras. Depois do mapeamento, o grafo guarda os resultados do raciocínio, mas não refaz o raciocínio sozinho. Se novos dados forem inseridos, é preciso executar o *reasoner* de novo ou escrever regras equivalentes no próprio banco.

2 QUESTÃO 2 – ALGORITMO DOS K-VIZINHOS MAIS PRÓXIMOS (KNN)

2.1 Base de dados

O algoritmo dos k-vizinhos mais próximos, conhecido pela sigla KNN (do inglês *k-nearest neighbors*), classifica um exemplo novo procurando na base os exemplos mais parecidos com ele e respondendo com a classe que aparece mais vezes entre esses exemplos (Cover; Hart, 1967). Não existe etapa de treinamento: a base inteira fica guardada e todo o trabalho é feito no momento da consulta. Por isso o KNN é chamado de método baseado em instâncias (Russell; Norvig, 2022). Aqui ele é aplicado à base de risco de crédito usada na primeira lista de exercícios.

A base tem cinco atributos de entrada e um atributo de classe. Os atributos de entrada são: história de crédito, dívida, garantia, renda e tempo de emprego. A classe é o risco. A base é uma versão ampliada do exemplo de risco de crédito apresentado por Luger (2013). A Tabela 1 mostra as 30 instâncias. Das 30, 9 são de risco alto, 12 de risco moderado e 9 de risco baixo.

Tabela 1 – Base de dados de risco de crédito

Ex.	História de crédito	Dívida	Garantia	Renda	Tempo de emprego	Risco
E1	Ruim	Alta	Nenhuma	$0 a $15k	Menos de 1 ano	Alto
E2	Desconhecida	Alta	Nenhuma	$15 a $35k	Menos de 1 ano	Alto
E3	Desconhecida	Baixa	Nenhuma	$15 a $35k	1 a 3 anos	Moderado
E4	Desconhecida	Baixa	Nenhuma	$0 a $15k	Menos de 1 ano	Alto
E5	Desconhecida	Baixa	Nenhuma	Acima de $35k	Acima de 3 anos	Baixo
E6	Desconhecida	Baixa	Adequada	Acima de $35k	Acima de 3 anos	Baixo
E7	Ruim	Baixa	Nenhuma	$0 a $15k	Menos de 1 ano	Alto
E8	Ruim	Baixa	Adequada	Acima de $35k	Acima de 3 anos	Moderado
E9	Boa	Baixa	Nenhuma	Acima de $35k	Acima de 3 anos	Baixo
E10	Boa	Alta	Adequada	Acima de $35k	Acima de 3 anos	Baixo
E11	Boa	Alta	Nenhuma	$0 a $15k	Menos de 1 ano	Alto
E12	Boa	Alta	Nenhuma	$15 a $35k	1 a 3 anos	Moderado
E13	Boa	Alta	Nenhuma	Acima de $35k	Acima de 3 anos	Baixo
E14	Ruim	Alta	Nenhuma	$15 a $35k	Menos de 1 ano	Alto
E15	Ruim	Alta	Adequada	$0 a $15k	Menos de 1 ano	Moderado

Ex.	História de crédito	Dívida	Garantia	Renda	Tempo de emprego	Risco
E16	Boa	Baixa	Adequada	$15 a $35k	1 a 3 anos	Baixo
E17	Desconhecida	Alta	Adequada	$15 a $35k	1 a 3 anos	Moderado
E18	Ruim	Baixa	Nenhuma	$15 a $35k	1 a 3 anos	Alto
E19	Boa	Baixa	Nenhuma	$0 a $15k	Menos de 1 ano	Moderado
E20	Desconhecida	Alta	Nenhuma	$0 a $15k	Menos de 1 ano	Alto
E21	Boa	Alta	Adequada	$15 a $35k	1 a 3 anos	Moderado
E22	Ruim	Alta	Adequada	Acima de $35k	Acima de 3 anos	Moderado
E23	Desconhecida	Baixa	Adequada	$0 a $15k	1 a 3 anos	Moderado
E24	Boa	Baixa	Nenhuma	$15 a $35k	Acima de 3 anos	Baixo
E25	Ruim	Baixa	Nenhuma	Acima de $35k	Menos de 1 ano	Moderado
E26	Desconhecida	Alta	Nenhuma	Acima de $35k	1 a 3 anos	Moderado
E27	Boa	Alta	Nenhuma	$0 a $15k	Acima de 3 anos	Moderado
E28	Desconhecida	Baixa	Nenhuma	$15 a $35k	Acima de 3 anos	Baixo
E29	Ruim	Alta	Nenhuma	$0 a $15k	1 a 3 anos	Alto
E30	Boa	Baixa	Adequada	Acima de $35k	1 a 3 anos	Baixo

Fonte: adaptado de Luger (2013).

2.2 Preparação dos dados

O KNN precisa medir a distância entre dois exemplos, e para isso os atributos precisam ser números. Todos os atributos da base são categóricos, então cada valor foi trocado por um número entre 0 e 1.

Renda e tempo de emprego têm uma ordem natural (pouco, médio, muito) e receberam os valores 0, 0,5 e 1. Dívida e garantia têm só dois valores e receberam 0 e 1. Na história de crédito, o valor "desconhecida" foi colocado entre "ruim" e "boa", porque um cliente sem histórico não é tão arriscado quanto um com histórico ruim nem tão seguro quanto um com histórico bom. O Quadro 4 resume a codificação.

Usar a mesma escala de 0 a 1 em todos os atributos é importante. Se um atributo tivesse valores muito maiores que os outros, ele dominaria o cálculo da distância e os demais quase não contariam.

Quadro 4 – Codificação numérica dos atributos

Atributo	Valor 0	Valor 0,5	Valor 1
História de crédito	Ruim	Desconhecida	Boa
Dívida	Alta	–	Baixa
Garantia	Nenhuma	–	Adequada
Renda	$0 a $15k	$15 a $35k	Acima de $35k
Tempo de emprego	Menos de 1 ano	1 a 3 anos	Acima de 3 anos

Fonte: elaborado pelo autor (2026).

2.3 Medida de distância

A distância usada é a euclidiana. Para dois exemplos, ela é a raiz quadrada da soma dos quadrados das diferenças entre os valores de cada atributo. Quanto menor a distância, mais parecidos são os exemplos. Dois exemplos idênticos têm distância zero.

O exemplo novo que será classificado é um cliente com história de crédito ruim, dívida alta, nenhuma garantia, renda acima de $35k e de 1 a 3 anos de emprego. Essa combinação não existe na base. Depois da codificação, o exemplo vira o vetor (0,0; 0,0; 0,0; 1,0; 0,5).

Como ilustração, o exemplo E1 da base é o vetor (0,0; 0,0; 0,0; 0,0; 0,0). A diferença está em dois atributos: renda (1 contra 0) e tempo de emprego (0,5 contra 0). A distância é a raiz de $1^2 + 0{,}5^2$, ou seja, a raiz de 1,25, que vale 1,118.

2.4 Passos do algoritmo

O algoritmo tem quatro passos. No primeiro, calcula-se a distância entre o exemplo novo e cada um dos 30 exemplos da base. No segundo, os exemplos são ordenados da menor para a maior distância. No terceiro, são selecionados os k primeiros, que são os k vizinhos mais próximos. No quarto, conta-se quantos vizinhos há de cada classe e a classe mais votada é a resposta.

A Tabela 2 mostra o resultado dos dois primeiros passos: as 30 distâncias, já em ordem crescente.

Tabela 2 – Distância do exemplo novo a cada instância da base, em ordem crescente

Posição	Exemplo	Vetor codificado	Distância	Risco
1	E26	(0,5; 0,0; 0,0; 1,0; 0,5)	0,500	Moderado
2	E14	(0,0; 0,0; 0,0; 0,5; 0,0)	0,707	Alto
3	E2	(0,5; 0,0; 0,0; 0,5; 0,0)	0,866	Alto
4	E29	(0,0; 0,0; 0,0; 0,0; 0,5)	1,000	Alto
5	E1	(0,0; 0,0; 0,0; 0,0; 0,0)	1,118	Alto
6	E12	(1,0; 0,0; 0,0; 0,5; 0,5)	1,118	Moderado
7	E13	(1,0; 0,0; 0,0; 1,0; 1,0)	1,118	Baixo
8	E18	(0,0; 1,0; 0,0; 0,5; 0,5)	1,118	Alto
9	E22	(0,0; 0,0; 1,0; 1,0; 1,0)	1,118	Moderado
10	E25	(0,0; 1,0; 0,0; 1,0; 0,0)	1,118	Moderado
11	E3	(0,5; 1,0; 0,0; 0,5; 0,5)	1,225	Moderado
12	E5	(0,5; 1,0; 0,0; 1,0; 1,0)	1,225	Baixo
13	E17	(0,5; 0,0; 1,0; 0,5; 0,5)	1,225	Moderado
14	E20	(0,5; 0,0; 0,0; 0,0; 0,0)	1,225	Alto
15	E28	(0,5; 1,0; 0,0; 0,5; 1,0)	1,323	Baixo
16	E7	(0,0; 1,0; 0,0; 0,0; 0,0)	1,500	Alto
17	E8	(0,0; 1,0; 1,0; 1,0; 1,0)	1,500	Moderado
18	E9	(1,0; 1,0; 0,0; 1,0; 1,0)	1,500	Baixo
19	E10	(1,0; 0,0; 1,0; 1,0; 1,0)	1,500	Baixo
20	E11	(1,0; 0,0; 0,0; 0,0; 0,0)	1,500	Alto
21	E15	(0,0; 0,0; 1,0; 0,0; 0,0)	1,500	Moderado
22	E21	(1,0; 0,0; 1,0; 0,5; 0,5)	1,500	Moderado
23	E27	(1,0; 0,0; 0,0; 0,0; 1,0)	1,500	Moderado
24	E4	(0,5; 1,0; 0,0; 0,0; 0,0)	1,581	Alto
25	E6	(0,5; 1,0; 1,0; 1,0; 1,0)	1,581	Baixo
26	E24	(1,0; 1,0; 0,0; 0,5; 1,0)	1,581	Baixo
27	E30	(1,0; 1,0; 1,0; 1,0; 0,5)	1,732	Baixo
28	E16	(1,0; 1,0; 1,0; 0,5; 0,5)	1,803	Baixo
29	E19	(1,0; 1,0; 0,0; 0,0; 0,0)	1,803	Moderado
30	E23	(0,5; 1,0; 1,0; 0,0; 0,5)	1,803	Moderado

Fonte: elaborado pelo autor (2026).

Com k = 3, os vizinhos são E26, E14 e E2. O primeiro é de risco moderado e os outros dois são de risco alto. Por dois votos a um, o exemplo novo é classificado como risco alto.

O resultado faz sentido. O cliente tem renda alta, mas tem história de crédito ruim, dívida alta e nenhuma garantia. Os clientes mais parecidos com ele na base, com essas mesmas características negativas, são na maioria de risco alto.

A Tabela 3 mostra a votação para quatro valores de k.

Tabela 3 – Votação dos vizinhos para diferentes valores de k

k	Votos em Alto	Votos em Moderado	Votos em Baixo	Classe prevista
1	0	1	0	Moderado
3	2	1	0	Alto
5	4	1	0	Alto
7	4	2	1	Alto

Fonte: elaborado pelo autor (2026).

2.5 Escolha do valor de k

A Tabela 3 mostra que a resposta depende de k. Com k = 1, o algoritmo olha só para o vizinho mais próximo, o exemplo E26, e responde risco moderado. Com k igual a 3, 5 ou 7, a resposta passa a ser risco alto.

Um k muito pequeno deixa o algoritmo sensível a exemplos isolados ou com erro: um único vizinho fora do padrão decide a resposta. Um k muito grande tem o problema contrário. Vizinhos distantes, que já não se parecem com o exemplo novo, entram na votação, e a classe mais comum da base passa a ganhar quase sempre. No limite, com k = 30, todo exemplo novo receberia a classe majoritária.

Outro cuidado é com empates. Costuma-se usar k ímpar para reduzir empates na votação, embora com três classes eles ainda possam acontecer. Nesta implementação, quando duas classes empatam, vence a que tem o vizinho mais próximo. Também há empates de distância: como os atributos têm poucos valores possíveis, vários exemplos ficam exatamente à mesma distância do exemplo novo. Na Tabela 2, por exemplo, vários exemplos aparecem com distância 1,118. Quando o corte dos k vizinhos cai no meio de um grupo empatado, a escolha de quem entra é feita pela ordem dos exemplos na base. Esse é um ponto fraco do KNN em bases pequenas e categóricas.

Uma variação comum é o voto ponderado, em que cada vizinho vota com peso maior quanto menor for a sua distância. Isso diminui o efeito dos empates e dos vizinhos distantes.

2.6 Avaliação

Para medir a qualidade do classificador foi usada a validação deixa-um-de-fora. Cada um dos 30 exemplos é retirado da base, classificado com os 29 restantes, e a resposta é comparada com a classe verdadeira. A Tabela 4 mostra o número de acertos para cada valor de k.

Tabela 4 – Acertos na validação deixa-um-de-fora

k	Acertos (de 30)	Acurácia
1	18	60,0 %
3	18	60,0 %
5	16	53,3 %
7	19	63,3 %
9	18	60,0 %

Fonte: elaborado pelo autor (2026).

A acurácia ficou entre 53,3 % e 63,3 %, com o melhor resultado em k = 7. O valor está bem acima do acerto ao acaso, que seria de cerca de 33 % com três classes, mas não é alto. Há três motivos para isso. A base é pequena. Os atributos têm poucos valores, o que gera muitos empates de distância. E a distância euclidiana trata os cinco atributos como igualmente importantes, quando na prática alguns pesam mais na definição do risco. As árvores de decisão construídas na primeira lista, por exemplo, colocaram o tempo de emprego na raiz, ou seja, trataram esse atributo como o mais importante.

3 QUESTÃO 3 – O CHATBOT ELIZA

3.1 Descrição e principais funcionalidades

O ELIZA é um dos primeiros programas de computador capazes de conversar com uma pessoa em linguagem natural. Foi criado por Joseph Weizenbaum no Instituto de Tecnologia de Massachusetts (MIT) e apresentado em artigo publicado em 1966 (Weizenbaum, 1966). A versão mais conhecida usa um roteiro chamado DOCTOR, que imita um psicoterapeuta da linha rogeriana. Nesse tipo de terapia o terapeuta fala pouco e devolve ao paciente o que ele mesmo disse, em forma de pergunta. Esse estilo foi escolhido porque dispensa conhecimento sobre o mundo: o programa pode responder "fale mais sobre isso" sem saber nada sobre o assunto.

O ELIZA recebe uma frase digitada pelo usuário e devolve outra frase. Ele não entende o significado do que foi escrito. O que ele faz é procurar palavras conhecidas na frase e aplicar regras de transformação de texto. As principais funcionalidades são as seguintes.

Identificação de palavras-chave. O programa procura na frase palavras que constam do roteiro, como "mãe", "sempre" ou "computador". Cada palavra-chave tem uma prioridade. Quando há mais de uma na frase, a de maior prioridade é usada primeiro.

Regras de decomposição. Cada palavra-chave tem um ou mais padrões que dividem a frase em partes. Um padrão pode ter trechos fixos e trechos livres, que aceitam qualquer sequência de palavras.

Regras de remontagem. Cada padrão de decomposição tem uma lista de modelos de resposta. O modelo indica onde encaixar as partes da frase do usuário. Quando o mesmo padrão é usado de novo, o programa passa para o modelo seguinte da lista, para não repetir sempre a mesma resposta.

Troca de pessoa. Antes de montar a resposta, o programa troca as palavras de primeira pessoa pelas de segunda, e vice-versa. Assim "eu" vira "você" e "meu" vira "seu". É isso que dá a impressão de que o programa está respondendo ao usuário.

Memória. Quando o usuário usa a palavra "meu" ou "minha", o programa guarda o trecho da frase. Mais tarde, se chegar uma frase sem nenhuma palavra-chave, ele retoma esse trecho guardado.

Respostas padrão. Se não há palavra-chave nem nada na memória, o programa usa uma frase genérica, como "por favor, continue".

Roteiros separados do programa. As palavras-chave e as regras ficam em um roteiro (*script*), fora do código. Trocando o roteiro, o mesmo programa conversa

sobre outro tema ou em outro idioma.

3.2 Trecho de diálogo

O roteiro original do ELIZA é em inglês. Para este trabalho foi escrita uma versão reduzida do programa em Python, com um roteiro em português de nove palavras-chave, seguindo o algoritmo descrito por Weizenbaum (1966). O diálogo do Quadro 5 foi produzido por essa versão. As falas do usuário foram digitadas pelo autor e as respostas são exatamente as geradas pelo programa.

Quadro 5 – Diálogo entre o usuário e a versão reduzida do ELIZA em português

Nº	Usuário	ELIZA
1	Olá, estou muito cansado ultimamente	Há quanto tempo você está muito cansado ultimamente?
2	Desde que comecei a faculdade, minha mãe sempre me cobra	Fale mais sobre a sua família.
3	Ela acha que eu não estudo o bastante	Você está dizendo não só para ser negativo?
4	Eu quero provar que ela está errada	Por que você quer provar que ela está errada?
5	Você não me entende	O que faz você pensar que eu não te entende?
6	Às vezes acho que só o computador me escuta	Computadores preocupam você?
7	Talvez	Antes você disse: "sua mãe sempre te cobra". Isso tem relação com o que estamos conversando?
8	Pode ser	Por favor, continue.
9	Tchau	Até logo. Obrigado por conversar comigo.

Fonte: elaborado pelo autor (2026).

3.3 Partes tecnicamente interessantes

Turno 1. A resposta reaproveita as palavras do usuário ("muito cansado ultimamente") dentro de uma pergunta. O efeito é o de alguém que ouviu e quer saber mais, embora o programa só tenha copiado o trecho que vinha depois da palavra "estou".

Turno 2. A frase tem duas partes separadas por vírgula. O programa descarta a primeira, que não tem palavra-chave, e trabalha só com a segunda. Nela há duas palavras-chave, "mãe" e "sempre". Como "mãe" tem prioridade maior, a resposta fala de família. Além disso, a palavra "minha" faz o trecho ser guardado na memória.

Turno 4. A troca de pessoa funciona bem: "eu quero provar" vira "você quer provar" e a pergunta fica natural.

Turno 7. A fala "talvez" não tem palavra-chave. Em vez de uma resposta genérica, o programa busca o que guardou no turno 2 e retoma o assunto da mãe, cinco turnos depois. Esse é o recurso que mais dá a impressão de que o programa acompanha a conversa.

3.4 Partes tecnicamente falhas

Turno 3. O usuário diz que a mãe acha que ele não estuda o bastante. A única palavra-chave encontrada é "não", e o programa pergunta se o usuário está sendo negativo. A resposta não tem relação com o que foi dito. O programa reagiu a uma palavra solta, sem considerar a frase.

Turno 5. A resposta sai com erro de concordância: "eu não te entende". O programa troca os pronomes, mas não sabe conjugar verbos. Em inglês esse problema quase não aparece, porque o verbo muda pouco de uma pessoa para outra. Em português ele é frequente.

Turno 6. O usuário fala de solidão ("só o computador me escuta"). Como "computador" é a palavra-chave de maior prioridade do roteiro, a resposta é sobre computadores e ignora o conteúdo emocional da frase.

Turno 7. A retomada da memória é interessante, mas o texto guardado é colado na resposta do jeito que está, e o encaixe nem sempre fica natural. O programa também não verifica se o assunto guardado ainda faz sentido naquele ponto da conversa.

Turno 8. Sem palavra-chave e com a memória vazia, resta a resposta padrão. Uma sequência de falas curtas do usuário levaria o programa a repetir essas frases genéricas, e a ilusão de conversa acabaria rápido.

De forma geral, o ELIZA não guarda o contexto da conversa (fora o recurso limitado da memória), não entende negação, ironia ou perguntas feitas a ele, e depende totalmente das palavras previstas no roteiro.

3.5 Regras e algoritmo

O roteiro é uma lista de palavras-chave. Cada uma tem uma prioridade, uma substituição opcional e um conjunto de regras. Cada regra é formada por um padrão de decomposição e pelos modelos de remontagem ligados a ele. Um exemplo do artigo original ajuda a entender. Para a frase "It seems that you hate me", o padrão de decomposição (0 YOU 0 ME) divide a frase em quatro partes: "It seems that", "you", "hate" e "me". O zero indica um trecho livre. O modelo de remontagem (WHAT MAKES YOU THINK I 3 YOU) usa a terceira parte, e a resposta é "What makes you think I hate you" (Weizenbaum, 1966).

O algoritmo executa os passos abaixo para cada frase do usuário.

Passo 1. A frase é lida da esquerda para a direita, palavra por palavra. Se aparece uma vírgula ou um ponto, o programa fica só com um dos lados: o trecho anterior, se nele já foi achada alguma palavra-chave, ou o trecho seguinte, se não foi.

Passo 2. Cada palavra é procurada no roteiro. As palavras-chave encontradas vão para uma pilha, ordenada pela prioridade. Durante a leitura também são feitas as substituições, como a troca de "eu" por "você".

Passo 3. O programa pega a palavra-chave do topo da pilha e testa os padrões de decomposição dela, na ordem em que estão no roteiro, sobre a frase já com as substituições.

Passo 4. Quando um padrão casa com a frase, o programa escolhe o próximo modelo de remontagem daquele padrão, encaixa as partes indicadas e devolve a resposta.

Passo 5. Se nenhum padrão da palavra-chave casa, o programa passa para a próxima palavra-chave da pilha.

Passo 6. Se a pilha está vazia, o programa usa um trecho da memória, se houver. Se não houver, usa uma resposta padrão.

O Quadro 6 mostra como esse algoritmo produziu cada resposta do diálogo do Quadro 5.

Quadro 6 – Regra aplicada em cada turno do diálogo

Nº	Palavra-chave usada	Frase após as substituições	Decomposição	Modelo de remontagem
1	estou (prioridade 2)	está muito cansado ultimamente	* está *	Há quanto tempo você está (2)?
2	mãe (prioridade 3)	sua mãe sempre te cobra	*	Fale mais sobre a sua família.
3	não (prioridade 0)	ela acha que você não estudo o bastante	*	Você está dizendo não só para ser negativo?
4	quero (prioridade 2)	você quer provar que ela está errada	* quer *	Por que você quer (2)?
5	você (prioridade 2)	eu não te entende	* eu * te *	O que faz você pensar que eu (2) te (3)?
6	computador (prioridade 5)	às vezes acha que só o computador te escuta	*	Computadores preocupam você?
7	nenhuma (memória)	talvez	–	Antes você disse: "(0)". Isso tem relação com o que estamos conversando?
8	nenhuma (resposta padrão)	pode ser	–	–
9	despedida	–	–	–

Fonte: elaborado pelo autor (2026).

No turno 1, a palavra-chave é "estou". A substituição troca "estou" por "está" e o padrão separa o que vem depois. Esse trecho ocupa a posição (2) do modelo. No turno 2, a pilha tem "mãe" (prioridade 3) e "sempre" (prioridade 1). A primeira vence e seu padrão aceita qualquer frase. No turno 3, a única palavra-chave é "não". No turno 4, a palavra-chave "quero" é trocada por "quer" e o resto da frase entra no modelo. No turno 5, "você" vira "eu" e "me" vira "te". O padrão separa "não" e "entende", que entram nas posições (2) e (3). No turno 6, "computador" tem a maior prioridade do roteiro. No turno 7 não há palavra-chave e a resposta vem da memória. No turno 8 não há palavra-chave nem memória, e sai a resposta padrão. No turno 9, a palavra "tchau" encerra a conversa.

3.6 Evolução: PARRY

O PARRY foi desenvolvido pelo psiquiatra Kenneth Colby na Universidade de Stanford, no início da década de 1970. Enquanto o ELIZA faz o papel do terapeuta, o PARRY faz o papel do paciente: ele simula uma pessoa com esquizofrenia paranoica (Colby; Weber; Hilf, 1971).

A principal novidade é que o PARRY tem um modelo interno. Ele mantém variáveis que representam o nível de medo, de raiva e de desconfiança, e esses

níveis mudam de acordo com o que o interlocutor diz. Uma pergunta sobre um tema sensível aumenta o medo. Um comentário que parece ofensa aumenta a raiva. A resposta escolhida depende desses níveis: com valores altos, o programa fica hostil, desconfiado ou se recusa a continuar.

O PARRY também tem um conjunto de crenças, entre elas um delírio de perseguição envolvendo a máfia, e conduz a conversa para esses temas. O ELIZA só reage. O PARRY tem assuntos próprios e uma atitude que se mantém ao longo da conversa.

Outra diferença é a forma de avaliação. Colby aplicou uma variação do teste de Turing: psiquiatras leram transcrições de entrevistas feitas com pacientes reais e com o PARRY e tentaram dizer qual era qual. O índice de acerto ficou perto do que se esperaria ao acaso (Colby *et al.*, 1972).

3.7 Evolução: AIML e ALICE

A AIML (Artificial Intelligence Markup Language) é uma linguagem baseada em XML criada por Richard Wallace a partir de 1995 para escrever o conhecimento de chatbots. O chatbot mais conhecido feito com ela é o ALICE (Artificial Linguistic Internet Computer Entity), também chamado de Alicebot (Wallace, 2009).

A unidade básica da AIML é a categoria. Cada categoria tem um padrão (*pattern*), que é comparado com a frase do usuário, e um modelo (*template*), que é a resposta. A ideia é a mesma dos pares de decomposição e remontagem do ELIZA, mas com vários recursos a mais.

Redução simbólica. A marca *srai* permite que uma categoria reenvie a frase, reescrita, para outra categoria. Com isso, várias formas de dizer a mesma coisa são reduzidas a uma forma única, que tem a resposta. O mesmo recurso serve para tratar sinônimos e para dividir uma frase longa em partes.

Contexto. A marca *that* faz a categoria valer só se a última fala do robô tiver sido uma frase específica. Isso permite interpretar um "sim" ou um "não" de acordo com a pergunta que o robô acabou de fazer. A marca *topic* agrupa categorias por assunto da conversa.

Variáveis. O robô pode guardar informações, como o nome do usuário, e usá-las depois.

Escala e eficiência. O ALICE chegou a ter cerca de 41 mil categorias, contra algumas dezenas de palavras-chave do ELIZA. Para procurar entre tantas categorias, os padrões são guardados em uma árvore, e a busca percorre a frase palavra por palavra.

Padrão aberto. A AIML é uma linguagem pública e o ALICE tem código aberto. Qualquer pessoa pode criar um robô novo escrevendo ou adaptando arquivos AIML, sem mexer no programa que os interpreta.

O ALICE venceu o Prêmio Loebner, uma competição baseada no teste de Turing, em 2000, 2001 e 2004. Mesmo assim, ele continua sendo um sistema de casamento de padrões: não aprende sozinho com as conversas e não entende o significado das frases. O Quadro 7 resume as diferenças entre os três sistemas. Nos três, o princípio é o mesmo: comparar a frase do usuário com padrões escritos por pessoas.

Quadro 7 – Comparação entre ELIZA, PARRY e ALICE

Aspecto	**ELIZA (1966)**	**PARRY (1972)**	**ALICE / AIML (1995)**
Autor	Joseph Weizenbaum (MIT)	Kenneth Colby (Stanford)	Richard Wallace
Papel simulado	Psicoterapeuta rogeriano	Paciente com esquizofrenia paranoica	Interlocutor de assuntos gerais
Conhecimento	Roteiro com palavras-chave e regras	Padrões, crenças e modelo de emoções	Milhares de categorias em AIML
Estado interno	Não tem	Medo, raiva e desconfiança	Variáveis e tópico da conversa
Contexto	Memória simples de trechos	Emoções mudam ao longo da conversa	Última fala do robô e tópico
Iniciativa	Só reage ao usuário	Leva a conversa para seus temas	Só reage ao usuário
Avaliação	Relatos de uso	Teste com psiquiatras	Prêmio Loebner

Fonte: elaborado pelo autor (2026).

4 QUESTÃO 4 – TIPOS DE AGENTES INTELIGENTES

4.1 Propriedades dos ambientes de tarefa

Um agente percebe o ambiente por meio de sensores e age sobre ele por meio de atuadores. Russell e Norvig (2022) organizam os programas de agentes em cinco categorias: reativo simples, reativo baseado em modelo, baseado em objetivo, baseado em utilidade e com aprendizagem, chamado aqui de adaptativo. O ambiente de tarefa de cada aplicação é descrito pela sigla PEAS, do inglês *Performance, Environment, Actuators, Sensors*: medida de desempenho, ambiente, atuadores e sensores.

Os ambientes são caracterizados por seis propriedades (Russell; Norvig, 2022).

Observável. O ambiente é totalmente observável quando os sensores dão acesso a tudo o que importa para escolher a ação. Caso contrário, é parcialmente observável.

Agentes. O ambiente é de agente único quando só há um agente decidindo. É multiagente quando há outros agentes cujo comportamento afeta o resultado, de forma competitiva ou cooperativa.

Determinístico. O ambiente é determinístico quando o próximo estado depende só do estado atual e da ação do agente. Quando há incerteza no resultado, é estocástico.

Episódico. O ambiente é episódico quando cada decisão é independente das anteriores. É sequencial quando uma decisão afeta as decisões futuras.

Estático. O ambiente é estático quando não muda enquanto o agente decide. Se muda, é dinâmico. É semidinâmico quando o ambiente não muda, mas o tempo gasto afeta o desempenho.

Discreto. O ambiente é discreto quando estados, percepções e ações formam um conjunto finito de valores distintos. Quando variam de forma contínua, como posição e velocidade, é contínuo.

4.2 Agente reativo simples

O agente reativo simples escolhe a ação olhando só para a percepção atual. Ele não guarda histórico. O comportamento é definido por regras do tipo condição e ação: se a condição é percebida, a ação é executada. Funciona bem quando a decisão correta depende só do que está sendo percebido naquele instante.

A primeira aplicação é o termostato de um ar-condicionado. A regra é: se a temperatura medida está acima da desejada, liga o compressor; se está abaixo, desliga. A segunda é a frenagem automática de emergência de um carro. A regra é: se há um obstáculo à frente e o tempo estimado até a colisão é menor que um limite, aciona os freios.

Quadro 8 – Descrição PEAS das aplicações de agente reativo simples

	Termostato de ar-condicionado	**Frenagem automática de emergência**
Desempenho	Manter a temperatura perto da desejada; gastar pouca energia	Evitar colisões; não frear sem necessidade
Ambiente	Sala, ar, pessoas, portas e janelas	Via, veículos, pedestres, clima
Atuadores	Compressor e ventilador (liga e desliga)	Sistema de freios, alerta sonoro
Sensores	Sensor de temperatura	Radar, câmera frontal, sensor de velocidade

Fonte: elaborado pelo autor (2026).

Quadro 9 – Propriedades dos ambientes das aplicações de agente reativo simples

Propriedade	**Termostato de ar-condicionado**	**Frenagem automática de emergência**
Observável	Parcialmente: mede a temperatura em um só ponto	Parcialmente: limitado ao alcance dos sensores
Agentes	Agente único	Multiagente: outros veículos e pedestres
Determinístico	Estocástico: portas abrem, pessoas entram	Estocástico
Episódico	Episódico: cada leitura decide sozinha	Episódico: cada leitura decide sozinha
Estático	Dinâmico	Dinâmico
Discreto	Contínuo	Contínuo

Fonte: elaborado pelo autor (2026).

4.3 Agente reativo baseado em modelo

O agente baseado em modelo mantém um estado interno, que é atualizado a cada percepção. Para isso ele usa um modelo de como o mundo muda e de como as suas ações afetam o mundo. Com o estado interno o agente consegue lidar com ambientes parcialmente observáveis, porque lembra o que não está vendo agora.

A primeira aplicação é o robô aspirador com mapeamento. Ele constrói um mapa da casa e marca as áreas já limpas, então sabe onde falta passar mesmo sem ver essas áreas. A segunda é o controle de cruzeiro adaptativo de um carro. Ele acompanha a posição e a velocidade do veículo da frente ao longo do tempo e

estima onde esse veículo estará no instante seguinte, para manter a distância segura.

Quadro 10 – Descrição PEAS das aplicações de agente baseado em modelo

	Robô aspirador com mapeamento	**Controle de cruzeiro adaptativo**
Desempenho	Área limpa; tempo e bateria gastos; não ficar preso	Manter a velocidade escolhida e a distância segura; conforto
Ambiente	Cômodos, móveis, sujeira, pessoas e animais	Rodovia, veículo da frente, demais veículos
Atuadores	Rodas, escovas, sucção	Acelerador e freio
Sensores	Sensor de distância, sensor de colisão, sensor de sujeira, odômetro	Radar, câmera, velocímetro

Fonte: elaborado pelo autor (2026).

Quadro 11 – Propriedades dos ambientes das aplicações de agente baseado em modelo

Propriedade	**Robô aspirador com mapeamento**	**Controle de cruzeiro adaptativo**
Observável	Parcialmente: só percebe o entorno próximo	Parcialmente: não conhece a intenção dos outros motoristas
Agentes	Agente único	Multiagente
Determinístico	Estocástico: rodas escorregam, objetos mudam de lugar	Estocástico
Episódico	Sequencial	Sequencial
Estático	Dinâmico	Dinâmico
Discreto	Contínuo	Contínuo

Fonte: elaborado pelo autor (2026).

4.4 Agente baseado em objetivo

O agente baseado em objetivo, além do estado, conhece uma descrição das situações que deseja alcançar. Ele escolhe as ações pensando no futuro: avalia que sequência de ações leva ao objetivo. Para isso usa busca e planejamento. É mais flexível que os agentes reativos, porque basta trocar o objetivo para mudar o comportamento.

A primeira aplicação é o planejador de rotas de um aplicativo de navegação. O objetivo é chegar ao endereço de destino, e o agente busca um caminho no mapa de ruas. A segunda é o robô de separação de pedidos em um armazém. O objetivo é levar a prateleira com o produto pedido até a estação de embalagem, e o agente planeja o trajeto entre as prateleiras.

Quadro 12 – Descrição PEAS das aplicações de agente baseado em objetivo

	Planejador de rotas (navegação)	**Robô de separação em armazém**
Desempenho	Chegar ao destino; rota válida	Pedido entregue na estação correta; sem colisões
Ambiente	Malha de ruas, trânsito, bloqueios	Galpão, prateleiras, outros robôs, pessoas
Atuadores	Tela e voz com as instruções de direção	Rodas, mecanismo de elevação da prateleira
Sensores	GPS, dados de trânsito, entrada do destino	Leitor de marcas no piso, sensores de distância, rede sem fio

Fonte: elaborado pelo autor (2026).

Quadro 13 – Propriedades dos ambientes das aplicações de agente baseado em objetivo

Propriedade	**Planejador de rotas (navegação)**	**Robô de separação em armazém**
Observável	Parcialmente: o trânsito à frente não é todo conhecido	Parcialmente
Agentes	Agente único (os outros motoristas são tratados como parte do ambiente)	Multiagente cooperativo
Determinístico	Estocástico	Estocástico
Episódico	Sequencial	Sequencial
Estático	Dinâmico	Dinâmico
Discreto	Discreto: o mapa é um grafo de ruas	Contínuo

Fonte: elaborado pelo autor (2026).

4.5 Agente baseado em utilidade

O objetivo só separa os estados em bons e ruins. Quando há várias formas de atingir o objetivo, ou objetivos que entram em conflito, é preciso uma medida mais fina. O agente baseado em utilidade usa uma função de utilidade, que dá uma nota para cada estado, e escolhe a ação que leva à maior utilidade esperada. Isso permite pesar critérios como tempo, custo, segurança e conforto.

A primeira aplicação é o táxi autônomo, exemplo clássico de Russell e Norvig (2022). Chegar ao destino é o objetivo, mas há rotas mais rápidas, mais seguras e mais baratas, e o agente precisa equilibrar esses critérios. A segunda é um sistema de recomendação de filmes. Ele estima o quanto o usuário vai gostar de cada filme e recomenda os de maior valor esperado.

Quadro 14 – Descrição PEAS das aplicações de agente baseado em utilidade

	Táxi autônomo	**Sistema de recomendação de filmes**
Desempenho	Segurança, rapidez, respeito às leis, conforto, lucro	Satisfação do usuário; filmes assistidos até o fim; retorno ao serviço
Ambiente	Ruas, trânsito, pedestres, clientes, clima	Usuários, catálogo de filmes, histórico de uso
Atuadores	Direção, acelerador, freio, setas, buzina, tela	Lista de recomendações na tela
Sensores	Câmeras, radar, GPS, velocímetro, sensores do motor	Cliques, avaliações, tempo assistido, buscas

Fonte: elaborado pelo autor (2026).

Quadro 15 – Propriedades dos ambientes das aplicações de agente baseado em utilidade

Propriedade	**Táxi autônomo**	**Sistema de recomendação de filmes**
Observável	Parcialmente	Parcialmente: o gosto do usuário não é visível
Agentes	Multiagente	Agente único
Determinístico	Estocástico	Estocástico
Episódico	Sequencial	Sequencial: uma recomendação influencia as próximas escolhas
Estático	Dinâmico	Dinâmico: catálogo e gostos mudam
Discreto	Contínuo	Discreto

Fonte: elaborado pelo autor (2026).

4.6 Agente adaptativo

O agente adaptativo, ou agente com aprendizagem, melhora o próprio desempenho com a experiência. Ele tem quatro componentes. O elemento de desempenho escolhe as ações, e pode ser qualquer um dos tipos anteriores. O crítico avalia como o agente está se saindo em relação a um padrão de desempenho. O elemento de aprendizagem usa essa avaliação para modificar o elemento de desempenho. O gerador de problemas sugere ações novas, que podem não ser as melhores no momento, mas ajudam o agente a descobrir coisas.

A primeira aplicação é o filtro de spam que aprende. O elemento de desempenho é o classificador de mensagens. O crítico é a resposta do usuário, quando ele marca uma mensagem como spam ou tira uma mensagem da caixa de spam. O elemento de aprendizagem reajusta o classificador com esses exemplos. A segunda é um programa de xadrez que aprende jogando contra si mesmo. O elemento de desempenho escolhe os lances. O crítico é o resultado de cada partida. O elemento de aprendizagem ajusta a avaliação das posições. O gerador de problemas faz o programa testar lances diferentes dos habituais.

Quadro 16 – Descrição PEAS das aplicações de agente adaptativo

	Filtro de spam que aprende	**Programa de xadrez que aprende por autojogo**
Desempenho	Spam bloqueado; mensagens legítimas não bloqueadas	Vitórias, empates e derrotas
Ambiente	Caixa de entrada, remetentes, usuário	Tabuleiro, peças, adversário, relógio
Atuadores	Mover para a caixa de spam, marcar, liberar	Lance jogado
Sensores	Texto, remetente e cabeçalhos da mensagem; correções do usuário	Posição das peças, lance do adversário, tempo restante

Fonte: elaborado pelo autor (2026).

Quadro 17 – Propriedades dos ambientes das aplicações de agente adaptativo

Propriedade	**Filtro de spam que aprende**	**Programa de xadrez que aprende por autojogo**
Observável	Parcialmente: a intenção do remetente não é visível	Totalmente
Agentes	Multiagente competitivo: quem envia spam muda de tática	Multiagente competitivo
Determinístico	Estocástico	Determinístico
Episódico	Episódico: cada mensagem é classificada separadamente	Sequencial
Estático	Estático durante a decisão; os padrões de spam mudam com o tempo	Semidinâmico: com relógio
Discreto	Discreto	Discreto

Fonte: elaborado pelo autor (2026).

4.7 Síntese

O Quadro 18 resume o que cada categoria usa para decidir e as aplicações apresentadas.

Quadro 18 – Síntese das categorias de agentes

Categoria	Como decide	Aplicações
Reativo simples	Regras de condição e ação sobre a percepção atual	Termostato; frenagem de emergência
Baseado em modelo	Regras sobre a percepção atual e o estado interno	Aspirador com mapeamento; cruzeiro adaptativo
Baseado em objetivo	Busca e planejamento para alcançar um objetivo	Planejador de rotas; robô de armazém
Baseado em utilidade	Escolha da ação de maior utilidade esperada	Táxi autônomo; recomendação de filmes
Adaptativo	Qualquer das formas anteriores, ajustada pela experiência	Filtro de spam; xadrez por autojogo

Fonte: elaborado pelo autor (2026).

5 QUESTÃO 5 – INFERÊNCIA EM GRANDES MODELOS DE LINGUAGEM

5.1 O vídeo e sua ideia central

Este resumo trata do vídeo "What is LLM inference? How AI models generate responses", publicado pelo canal IBM Technology (IBM Technology, 2026). O vídeo tem cerca de sete minutos e explica o que acontece entre o momento em que uma pessoa envia uma mensagem a um grande modelo de linguagem (LLM, do inglês *large language model*) e o momento em que a resposta aparece na tela.

O ponto de partida é uma comparação. Treinar um modelo de ponta pode custar centenas de milhões de dólares e levar meses. Quando o treinamento termina, porém, o modelo é só um conjunto enorme de números parado na memória. Sozinho, ele não sabe fazer nada. A pergunta que o vídeo se propõe a responder é o que acontece nos bastidores para dar vida a esse modelo. A resposta é um processo chamado inferência.

Segundo o vídeo, a inferência está por trás da maior parte dos chatbots, dos assistentes de programação e das aplicações com agentes usados hoje. Ou seja, quase tudo o que o público conhece como inteligência artificial generativa é, na prática, um modelo já treinado sendo executado muitas vezes.

5.2 Treinamento e inferência

O vídeo descreve o modelo treinado como uma pilha estática de pesos. Os pesos são números especializados, ajustados durante o treinamento para representar informação. Depois do treinamento eles não mudam mais.

A inferência é definida como o processo de pegar esse modelo treinado e executá-lo sobre uma entrada nova para produzir algum tipo de saída. No treinamento, o modelo aprende. Na inferência, ele usa o que aprendeu.

A diferença mais importante entre os dois, do ponto de vista prático, é a frequência. O treinamento acontece uma vez. A inferência acontece toda vez que alguém envia uma mensagem ao modelo. Um serviço popular recebe um número muito grande de mensagens por dia, e cada uma delas dispara uma nova execução.

Dessa diferença sai uma afirmação que o próprio vídeo trata como surpreendente: ao longo do tempo, a inferência costuma custar mais que o treinamento. O treinamento é caro, mas é pago uma vez. A inferência é mais barata por execução, mas o custo se repete a cada requisição e não para de crescer enquanto o modelo estiver em uso.

A consequência é que, com o passar do tempo, fica cada vez mais importante servir os modelos da forma mais eficiente possível. O vídeo anuncia então seu objetivo: mostrar o que realmente acontece entre o *prompt* e a resposta e como tornar esse processo o mais rápido e barato possível.

5.3 Motor de inferência

O software que gerencia a inferência recebe o nome de motor de inferência (*inference engine*). O trabalho dele é pegar a pilha de pesos e transformá-la em algo que atenda requisições de forma confiável, rápida e em escala.

O vídeo detalha o que isso significa em três tarefas. A primeira é organizar a fila das requisições que chegam. A segunda é gerenciar os recursos das unidades de processamento gráfico (GPUs), que são o hardware onde o modelo roda. A terceira é decidir como juntar o trabalho de várias requisições para processá-lo em conjunto.

Como exemplos de motores de inferência, o vídeo cita o vLLM, o Ollama e o llama.cpp. São ferramentas conhecidas de quem executa modelos por conta própria, seja em servidores, seja no próprio computador.

5.4 Formato da requisição

A partir desse ponto o vídeo acompanha uma mensagem desde o envio. O primeiro aspecto é o formato em que ela é enviada.

A maioria dos motores de inferência oferece um ponto de acesso (*endpoint*) compatível com o padrão da OpenAI. Isso quer dizer que o mesmo formato de requisição pode ser usado para enviar a consulta, não importa qual modelo esteja rodando por trás.

O vídeo destaca a utilidade disso para quem desenvolve aplicações. Uma aplicação construída para chamar um modelo pode trocá-lo por outro com facilidade, sem que o código precise ser reescrito. Basta apontar para outro endereço ou indicar outro nome de modelo.

5.5 Tokenização

Antes mesmo de a requisição entrar no *data center*, o texto precisa ser traduzido para um formato que o modelo entenda. Esse processo é a tokenização.

A tokenização quebra o texto em *tokens*. Um *token* pode ser uma palavra inteira, só um pedaço de uma palavra ou um sinal de pontuação. Cada *token* é associado a um número de identificação arbitrário. O vídeo faz questão de dizer que

o modelo nunca vê as palavras de fato. Ele vê apenas a sequência de números.

Essa etapa quase não exige poder de processamento. Mesmo assim ela é importante, porque define quanto o *prompt* vai custar: a cobrança é feita por *token*. Um texto que gera mais *tokens* custa mais, e isso vale tanto para o que o usuário envia quanto para o que o modelo responde.

O vídeo acrescenta uma observação sobre o tamanho do vocabulário, que é o conjunto de todos os *tokens* que o modelo conhece. Modelos com vocabulário maior conseguem representar o texto de forma mais eficiente, usando menos *tokens* por *prompt*. Em compensação, um vocabulário maior aumenta o esforço computacional do treinamento. Há, portanto, um equilíbrio a ser escolhido por quem projeta o modelo.

5.6 Autenticação e roteamento

A requisição precisa ser processada em hardware físico. Na maioria dos casos, esse hardware fica em um *data center*.

Ao chegar ao *data center*, a requisição é autenticada. Isso é feito por meio de uma chave de API ou de um *token* de acesso, que é algo que comprova que o usuário tem permissão para usar aqueles recursos de GPU. Aqui a palavra *token* tem outro sentido: é uma credencial, e não um pedaço de texto.

Depois da autenticação vem o roteamento. Um componente chamado roteador de inferência examina o *prompt*: olha o tamanho dele e o conteúdo que ele traz. Com base nisso, encaminha a requisição para o hardware disponível que for mais adequado para ela naquele momento.

5.7 Agrupamento de requisições

O vídeo lembra que uma requisição nunca chega sozinha. Muitas outras chegam ao mesmo tempo. Passar cada uma delas individualmente pelo modelo seria um desperdício.

Para ilustrar, o vídeo usa uma comparação: seria como uma padaria usar um forno industrial para assar um doce de cada vez. O forno ficaria quase vazio a maior parte do tempo. No caso do modelo, isso desperdiçaria uma quantidade enorme de recursos de GPU e derrubaria a vazão (*throughput*) do sistema, que é a quantidade de requisições que ele consegue atender de uma vez.

A solução é o agrupamento em lotes (*batching*). As requisições são reunidas para que a GPU as processe como uma única unidade de trabalho, maior e mais eficiente.

O agrupamento simples tem um problema: as requisições não levam o mesmo tempo para ser processadas. Uma resposta curta termina logo e uma resposta longa demora. Se o lote só pudesse ser trocado quando todas terminassem, parte da GPU ficaria parada esperando a requisição mais demorada.

Por isso os motores modernos usam o agrupamento contínuo (*continuous batching*). Assim que uma requisição termina, outra é colocada no lugar dela, sem esperar o restante do lote. Dessa forma as GPUs ficam perto da utilização total o tempo todo.

Quem cuida desse processo é um escalonador (*scheduler*). Ele monta os lotes e leva as requisições até o hardware onde a computação de fato acontece e a resposta é gerada.

5.8 Fase de prefill

A parte central do vídeo é a explicação da última etapa, a geração da resposta. A inferência em um LLM tem duas fases distintas: *prefill* e *decode*.

Quando a requisição chega ao modelo, ela entra na fase de *prefill*. Nessa fase o modelo lê o *prompt* inteiro de uma só vez e constrói uma compreensão dele.

O resultado dessa leitura é guardado em uma estrutura chamada *cache* de chaves e valores (*key-value cache*, ou *KV cache*). O vídeo a descreve como uma memória corrente de tudo o que o modelo leu até ali.

O vídeo explica por que esse *cache* é necessário. Sem ele, o modelo teria que reprocessar a conversa inteira de novo para cada *token* que quisesse produzir. É o *KV cache* que evita que as conversas fiquem muito mais lentas à medida que ficam mais longas.

Quanto ao desempenho, o *prefill* é limitado pela capacidade de computação (*compute-bound*). Ele exige grandes rajadas de cálculo em paralelo, e sua velocidade depende de quanta computação bruta o hardware consegue sustentar ao mesmo tempo. Como todos os *tokens* do *prompt* já estão disponíveis, eles podem ser processados juntos.

5.9 Fase de decode

Terminado o *prefill*, o *KV cache* é passado para a fase de *decode*. É nela que a resposta do modelo é gerada, um *token* de cada vez.

Em cada passo do *decode*, o modelo executa uma passagem direta (*forward pass*) e produz uma distribuição de probabilidade sobre todos os *tokens* do seu vocabulário. O vídeo traduz isso de forma simples: é uma lista ordenada de quais

tokens poderiam vir em seguida.

Um *token* é então selecionado a partir dessa distribuição e acrescentado ao final da sequência. Depois disso o processo inteiro começa de novo, para gerar o *token* seguinte. O ciclo se repete até a resposta terminar.

Duas dependências tornam essa fase diferente da anterior. Cada *token* depende dos que vieram antes dele, então não é possível gerar vários ao mesmo tempo. Além disso, em cada passo o modelo precisa consultar o *KV cache* inteiro para lembrar o que aprendeu no *prefill*.

Por causa disso, o *decode* é limitado pela largura de banda da memória (*memory-bandwidth-bound*). A velocidade dele não depende de quanta conta o hardware consegue fazer, e sim da rapidez com que os dados conseguem sair da memória.

O vídeo resume a diferença entre as fases em uma frase: o *prefill* quer computação bruta, enquanto o *decode* quer acesso rápido à memória. O Quadro 19 organiza essa comparação.

Quadro 19 – Comparação entre as fases de *prefill* e *decode*

Aspecto	***Prefill***	***Decode***
O que faz	Lê o *prompt* inteiro de uma vez	Gera a resposta, um *token* por vez
Resultado	*KV cache* com o que foi lido	*Tokens* da resposta
Forma de processamento	Em paralelo, sobre todos os *tokens* do *prompt*	Em sequência, cada *token* depende dos anteriores
Uso do *KV cache*	Constrói o *cache*	Consulta o *cache* a cada passo
Gargalo	Capacidade de computação	Largura de banda da memória
Métrica associada	Tempo até o primeiro *token* (TTFT)	Latência entre *tokens* (ITL)

Fonte: elaborado pelo autor (2026), com base em IBM Technology (2026).

5.10 Métricas de latência

O vídeo mostra que as duas fases dão origem a duas medidas muito comuns em avaliações de desempenho.

A primeira é o tempo até o primeiro *token* (*time to first token*, TTFT). É o tempo que passa até o usuário receber o primeiro *token* da resposta. Na prática, essa medida mostra quanto tempo a fase de *prefill* está levando.

A segunda é a latência entre *tokens* (*inter-token latency*, ITL). É o tempo entre a chegada de um *token* e a chegada do seguinte. Essa medida mostra quanto tempo cada passo da fase de *decode* está levando.

Juntas, as duas medidas formam a maior parte do que as pessoas querem dizer quando falam da latência de um LLM. A ligação entre fases e métricas tem

utilidade prática. Um TTFT alto indica que o problema está no *prefill*, o que pode acontecer com *prompts* muito longos. Uma ITL alta indica que o problema está no *decode*.

5.11 Síntese do processo

No encerramento, o vídeo retoma as ideias principais. A inferência é o modo como um modelo treinado e estático de fato gera uma resposta. Cada requisição é organizada em lotes, para manter o hardware ocupado mesmo quando as requisições têm tamanhos diferentes. As requisições são encaminhadas ao hardware e passam por uma fase de *prefill*, limitada pela computação, e por uma fase de *decode*, limitada pela largura de banda da memória. Reunido, esse conjunto é a maquinaria que fica entre um modelo treinado e um produto que as pessoas conseguem usar em tempo real. O Quadro 20 reúne as etapas na ordem em que acontecem.

Quadro 20 – Etapas entre o envio do *prompt* e a resposta

Etapa	O que acontece
1. Envio	A aplicação envia a requisição em um formato padrão, compatível com vários modelos
2. Tokenização	O texto é dividido em *tokens* e cada um recebe um número de identificação
3. Autenticação	O *data center* confere a chave de API ou o *token* de acesso
4. Roteamento	O roteador de inferência escolhe o hardware mais adequado para a requisição
5. Agrupamento	O escalonador junta a requisição a outras em um lote contínuo
6. *Prefill*	O modelo lê o *prompt* inteiro e monta o *KV cache*
7. *Decode*	O modelo gera a resposta *token* por *token*, consultando o *KV cache*

Fonte: elaborado pelo autor (2026), com base em IBM Technology (2026).

Por ser curto e introdutório, o vídeo deixa alguns assuntos de fora. Ele não explica como o *token* é escolhido a partir da distribuição de probabilidade, nem o que acontece dentro do modelo durante a passagem direta, nem outras técnicas de otimização além do agrupamento e do *KV cache*. Ainda assim, ele cumpre o objetivo de dar uma visão completa e ordenada do caminho entre o *prompt* e a resposta.

REFERÊNCIAS

COLBY, Kenneth Mark *et al.* Turing-like indistinguishability tests for the validation of a computer simulation of paranoid processes. **Artificial Intelligence**, Amsterdam, v. 3, p. 199-221, 1972.

COLBY, Kenneth Mark; WEBER, Sylvia; HILF, Franklin Dennis. Artificial paranoia. **Artificial Intelligence**, Amsterdam, v. 2, n. 1, p. 1-25, 1971.

COVER, Thomas; HART, Peter. Nearest neighbor pattern classification. **IEEE Transactions on Information Theory**, New York, v. 13, n. 1, p. 21-27, 1967.

GRUBER, Thomas R. A translation approach to portable ontology specifications. **Knowledge Acquisition**, London, v. 5, n. 2, p. 199-220, 1993.

HOGAN, Aidan *et al.* Knowledge graphs. **ACM Computing Surveys**, New York, v. 54, n. 4, p. 1-37, 2021.

IBM TECHNOLOGY. **What is LLM inference?** How AI models generate responses. [*S. l.*: *s. n.*], 2026. 1 vídeo (7 min). Disponível em: https://www.youtube.com/watch?v=615-yWyS1n0. Acesso em: 11 nov. 2026.

LUGER, George F. **Inteligência artificial**. 6. ed. São Paulo: Pearson, 2013.

PATEL-SCHNEIDER, Peter F.; MOTIK, Boris (ed.). **OWL 2 Web Ontology Language**: mapping to RDF graphs. 2. ed. [*S. l.*]: W3C, 2012. Disponível em: https://www.w3.org/TR/owl2-mapping-to-rdf/. Acesso em: 11 nov. 2026.

RUSSELL, Stuart; NORVIG, Peter. **Inteligência artificial**: uma abordagem moderna. 4. ed. Rio de Janeiro: GEN LTC, 2022.

W3C OWL WORKING GROUP. **OWL 2 Web Ontology Language**: document overview. 2. ed. [*S. l.*]: W3C, 2012. Disponível em: https://www.w3.org/TR/owl2-overview/. Acesso em: 11 nov. 2026.

WALLACE, Richard S. The anatomy of A.L.I.C.E. *In*: EPSTEIN, Robert; ROBERTS, Gary; BEBER, Grace (ed.). **Parsing the Turing test**: philosophical and methodological issues in the quest for the thinking computer. Dordrecht: Springer, 2009. p. 181-210.

WEIZENBAUM, Joseph. ELIZA: a computer program for the study of natural language communication between man and machine. **Communications of the ACM**, New York, v. 9, n. 1, p. 36-45, 1966.